Multilayer Networks

Machine Learning with Python — Lecture 2, Session 2 (Oct 20)

EUGLOH — *Problem Solving Using Open-Source Languages; R and Python*

University of Novi Sad

Today's session

Multilayer networks

- The **big idea** — stacking neurons to learn a useful transformation
- The **MLP** — layers, shapes and matrix products
- The **forward pass** — and what a hidden layer does to the data

Training a network

- The **cross-entropy** loss and the **backward pass** (backpropagation)
- **Gradient descent**, the **learning rate**, and an MLP **from scratch**
- Weight initialization, **optimizers**, regularization and **capacity**

Outputs

- **Multi-class** classification with **softmax**

Session 2 of 4 today. (The sigmoid, activation functions and the XOR problem were covered in Session 1.)

Part 1 — Multilayer networks

The big idea

- A single neuron is a **linear classifier** — it needs linearly separable data and cannot solve **XOR** (Session 1).
- What if we could **transform** the data into a representation where it *becomes* linearly separable?
- And who computes that transformation? **Another neuron!**
- Stack layers of neurons → a **multi-layer perceptron (MLP)**.

The layers learn the transformation; the last layer separates. Everything is trained end-to-end with one algorithm: **backpropagation**.

The multi-layer perceptron (MLP)

Layers of neurons: **input → hidden → output**. For one hidden layer, each hidden unit takes a weighted sum of the inputs and applies the activation:

$$\text{hidden} = \phi(W_1 x + b_1), \qquad \hat{p} = \sigma(W_2 \cdot \text{hidden} + b_2)$$

Every layer simply feeds its output to the next, so the whole network is a chain of weighted sums and activations.

That chain is **matrix multiplication** — the reason GPUs are so good at this. (One hidden layer with enough units can approximate *any* continuous function, on a bounded region — but nobody tells you how many units; you learn that from the data.)

The forward pass, step by step

Feed an input x through the network:

$$z^{(1)} = W_1 x + b_1, \quad a^{(1)} = \phi(z^{(1)}), \quad z^{(2)} = W_2 a^{(1)} + b_2, \quad \hat{p} = \sigma(z^{(2)})$$

- Each line is a **weighted sum (+ bias)** followed by the **activation** — exactly the single neuron from Session 1.
- A whole batch is one matrix product: stack the inputs into X and compute $Z^{(1)} = XW_1 + b_1$.
- In plain words: **signals flow forward**, layer by layer, from inputs to prediction.

What a hidden layer does — see it live

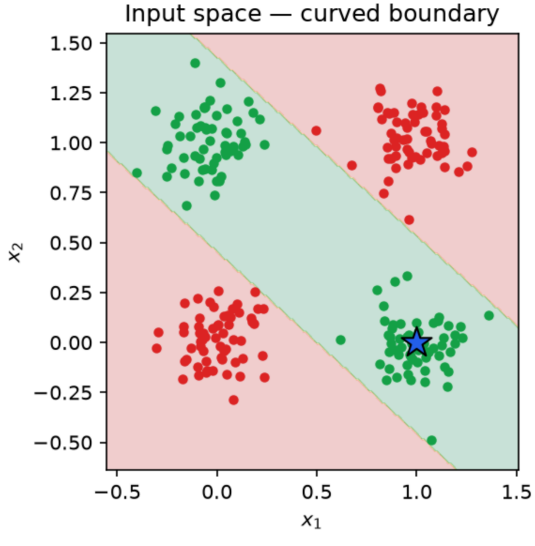
The same **XOR** data, shown in two spaces. On the left, the original input space: the network's boundary has to bend, so no single line works. On the right, the *same* points after one hidden layer ($\phi = \tanh$): they have been **pulled apart into a linearly separable arrangement**.

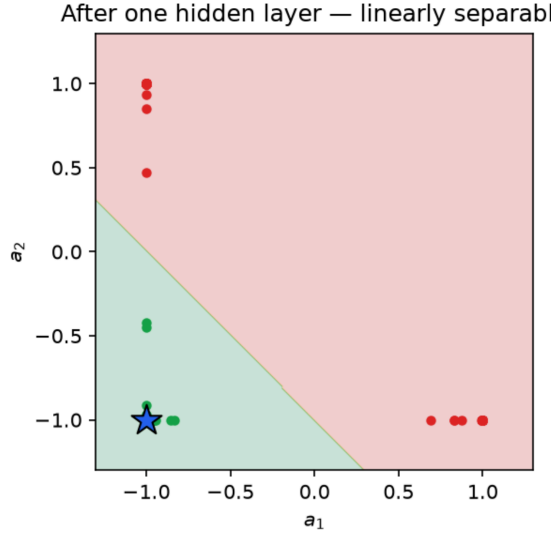
Drag the sliders to move a point through the network and watch its hidden activation — that is the **forward pass**. (The network is trained on standardised inputs internally; the picture shows the raw measurements, so the sliders range over the unit square.)

input x_1 1

input x_2 0

forward pass: hidden activation = (-1.00, -1.00) → output $\hat{p}$ = 1.00





Part 2 — Training a network

Training a network — loss and gradients

The recipe carries over from logistic regression: pick a **loss**, then descend its **gradient**. For classification the **cross-entropy** is

$$\mathcal{L} = -\frac{1}{n}\sum_{i}\Big[\, y_i \log \hat{p}_i + (1 - y_i)\log(1 - \hat{p}_i)\,\Big]$$

- small when the prediction matches the true label, large when it is **confidently wrong**;
- for a single **logistic** neuron it is **convex** — for a whole network it is **not**, so we settle for a good local minimum;
- and, crucially, its gradient is a **simple** function of the error $\hat{p} - y$.

What is new is computing that gradient *through* several layers — the **backward pass**.

The backward pass — backpropagation

Work **backwards** from the output, applying the **chain rule** layer by layer:

- **Output layer** — exactly like logistic regression: the "error" is $\hat{p} - y$.
- **Hidden layer** — push that error back through the output weights, then multiply by the derivative of the activation. (For $\tanh$ that derivative is simply $1 - a^2$, where a is the activation.)
- **Repeat** — each layer's gradient is built from the *next* layer's error, so every step is only local arithmetic.

The recipe:

1. **Initialise** the parameters (small random weights)
2. **Forward pass** — compute all activations
3. **Backward pass** — gradients with the chain rule
4. **Update** all parameters with gradient descent
5. **Repeat!**

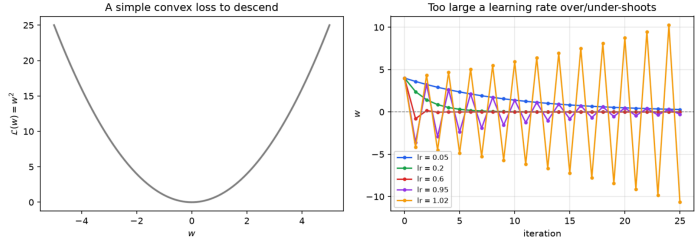
In plain words: **blame flows backwards** — each weight is told how much it contributed to the final error. There is only **one loop** (the epochs); everything inside is matrix multiplication.

Optimization — how we actually descend

Backpropagation gives us the **gradients**; **optimization** decides how to use them:

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}$$

- η, the **learning rate**, is *the* single most important hyperparameter — too small crawls, too large **diverges**.
- one pass over the whole training set is an **epoch**; batch / mini-batch / stochastic gradient descent trade step quality against cost — mini-batches are the workhorse of deep learning.

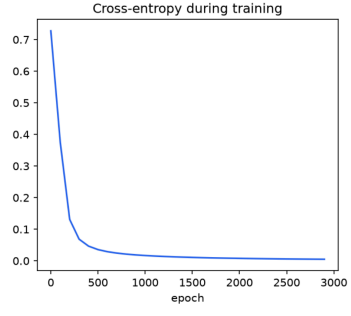


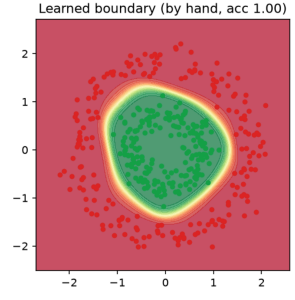
An MLP by hand, in numpy

2 inputs → 16 hidden units ($\tanh$) → 1 output (sigmoid), trained with **full-batch gradient descent** on the circles dataset (standardised). Forward pass, backward pass, update — a handful of lines:

```
for _ in range(iters):
    A1 = tanh(X @ W1 + b1);  P = sigmoid(A1 @ W2 + b2)   # forward
    d2 = (P - y) / n;  dW2 = A1.T @ d2                   # backward
    d1 = (d2 @ W2.T) * (1 - A1**2);  dW1 = X.T @ d1
    W2 -= lr * dW2;  W1 -= lr * dW1                       # update
```

It drew a **round** boundary that no linear model could — exactly the shape the data demands.





The same in scikit-learn

```
from sklearn.neural_network import MLPClassifier
mlp = MLPClassifier(hidden_layer_sizes=(16,), activation="tanh",
                    max_iter=2000, random_state=0).fit(X, y)
```

Same network, same data — but scikit-learn swaps our plain gradient descent for **Adam** (a smarter variant), and adds **early stopping** and **regularization** out of the box. Everything we built by hand is one line.

Part 3 — Training in practice

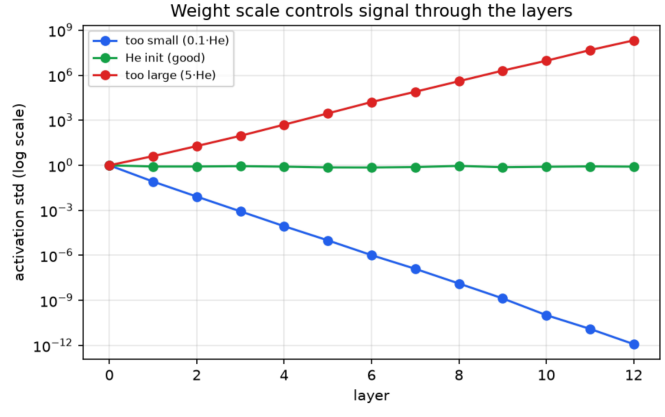
Weight initialization

You cannot start all weights at **zero** — every neuron would be identical and stay identical (no **symmetry breaking**). But *how* we randomise matters:

- **too small** → activations shrink layer by layer → **vanishing** signal, no learning;
- **too large** → activations blow up → **exploding** signal.

The fix is to scale the weights by the layer's **fan-in** — **He initialisation** sets the standard deviation to $\sigma = \sqrt{2/n_{\text{in}}}$ (Xavier is similar, for $\tanh$).

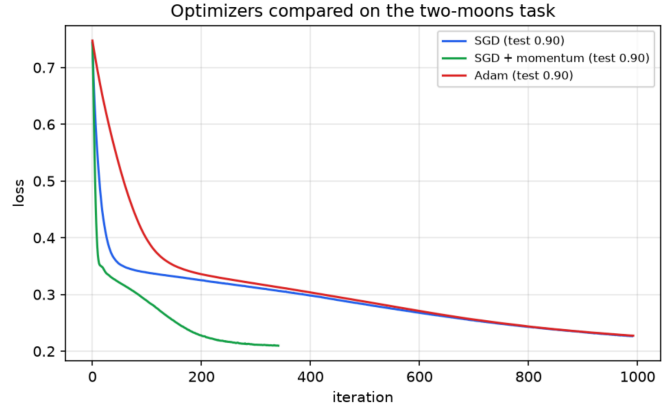
scikit-learn does this for you; in PyTorch it is `nn.init.kaiming_normal_`.



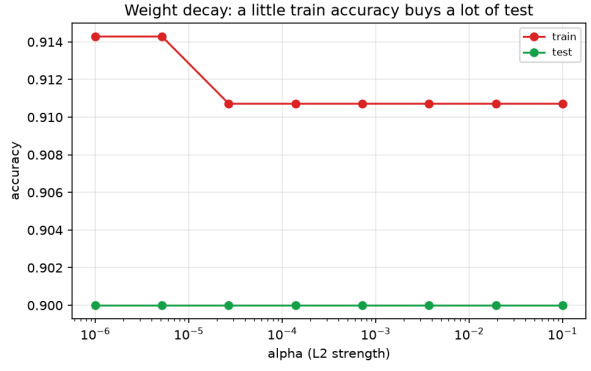
Optimizers — beyond plain gradient descent

Plain gradient descent is the starting point; three famous upgrades:

- **SGD** — estimate the gradient on a **mini-batch** instead of the whole dataset: noisier, far cheaper per step.
- **Momentum** — carry part of the previous step forward, like a ball rolling downhill: $v \leftarrow \beta v - \eta \nabla \mathcal{L}$, then $w \leftarrow w + v$.
- **Adam** — the default in practice. It tracks a running average of the gradient and of its magnitude, adapting the learning rate **per weight** — forgiving of a badly chosen η.



Regularization — keeping the network honest



A flexible network can **memorise** the training set. Regularization adds a cost for complexity so the model prefers simpler solutions:

- **Weight decay (L2)** — penalise large weights; `MLPClassifier(alpha=...)` (larger = stronger), the same penalty as `Ridge`.
- **Dropout** — randomly switch off neurons during training so the network cannot rely on any single one.
- **Early stopping** — stop when the **validation** loss stops improving: `MLPClassifier(early_stopping=True)`.

You trade a little **training** accuracy for a lot of **test** accuracy.

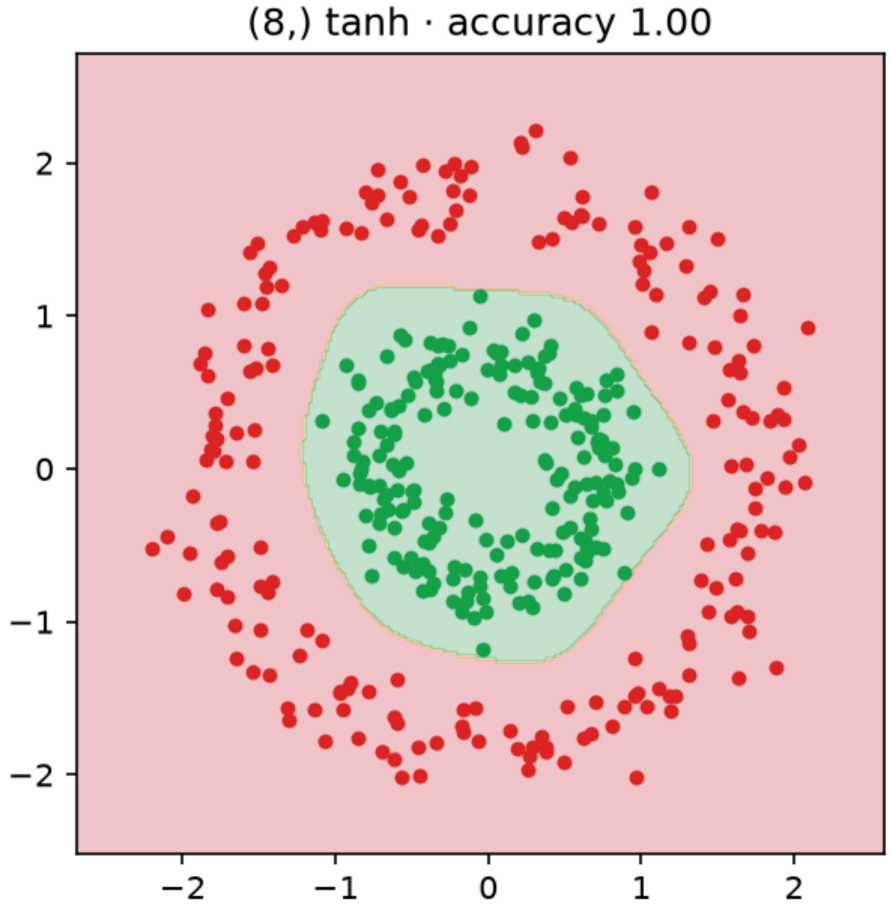
Capacity — width and depth

- **Width** — more units in a hidden layer.
- **Depth** — more hidden layers, each building on the previous one (edges → parts → objects, in vision).

More capacity fits more complex boundaries — and **overfits** sooner. Drag the slider and swap the activation: a single hidden unit cannot even bend the boundary.

hidden units 8

activation tanh



Training in practice

Getting a network to train is a **craft**. The reliable recipe:

1. **Split first** — train / validation / test. Never tune on the test set.
2. **Scale the inputs** — networks are very sensitive to feature scales.
3. **Start small** — a tiny network, and a baseline (`LogisticRegression`).
4. **Watch both curves** — training loss *and* validation loss.
5. **Change one thing at a time** — learning rate, width, regularization.
6. Only then **scale up** — more data, more depth, more epochs.

symptom	likely cause	fix
loss barely moves	lr too small, unscaled features	raise lr, standardise
loss explodes / `NaN`	lr too large	lower it ~10×, standardise
train great, validation poor	**overfitting**	more data, `alpha`, dropout, early stopping
both poor	**underfitting**	bigger network, train longer
stuck at the majority class	class imbalance	class weights, resampling, better metric

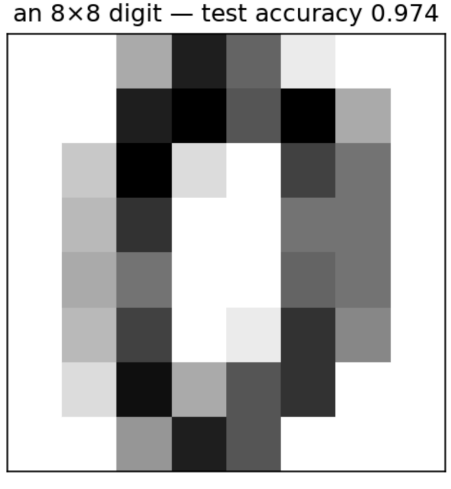
Part 4 — Outputs

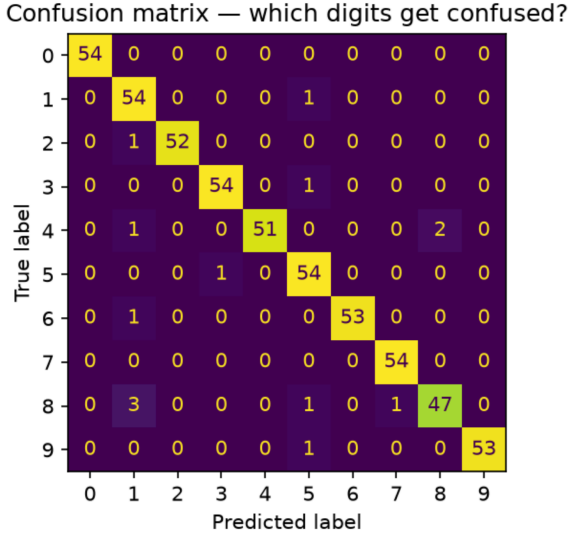
Multi-class: softmax output

With K classes the last layer has K neurons; **softmax** turns their raw scores into a probability distribution:

$$\hat{p}_k = \frac{e^{zk}}{\sum_{j=1}^{K} e^{zj}}, \qquad \sum_k \hat{p}_k = 1$$

Exponentiate each score (to make it positive), then divide by the total. The loss is the multi-class cross-entropy — `MLPClassifier` handles all of it. Demo: 8×8 handwritten digits, 10 classes.





Summary

- A single neuron is **linear**; a **hidden layer** transforms the inputs so the last layer can separate them — that is the **MLP**.
- The **forward pass** sends the signal through the network; the **backward pass** (backpropagation) uses the **chain rule** to get every gradient.
- **Gradient descent** turns gradients into updates; the **learning rate** is the crucial knob. We built the whole thing **from scratch** in a few lines.
- Training in practice: **initialization**, **optimizers**, **regularization** and **capacity** are the levers — and we watch the curves.
- Many classes use the **softmax** output layer.

Next session: **beyond the MLP** — convolutional networks, transformers and autoencoders.